

BURSA TEKNİK ÜNİVERSİTESİ

Mühendislik ve Doğa Bilimleri Fakültesi — Bilgisayar Mühendisliği Bölümü

BİLGİSAYAR AĞLARI DERSİ — DÖNEM PROJESİ

NetProbe

UDP Tabanlı Güvenilir Dosya Aktarımı, Trafik İzleme ve Ağ Performans Analiz Platformu

Teknik Rapor

Alan	Bilgi
Proje Başlığı	NetProbe — UDP Tabanlı Güvenilir Dosya Aktarımı
Ders	Bilgisayar Ağları
Programlama Dili	Python 3 (standart kütüphane: socket, threading, struct, zlib, hashlib)
Aktarım Modları	Stop-and-Wait + Sliding Window (Selective Repeat)
Grup Üyeleri	Hüseyin Acar – 23360859070 Kerem Özcan - 23360859718
Tarih	Haziran 2026
GitHub	https://github.com/RetcapS/NetProbe

İçindekiler

1. Giriş
 2. Problem Tanımı
 3. Sistem Mimarisi
 4. Protokol Tasarımı
 5. Gerçekleme Detayları
 6. Deney Ortamı
 7. Performans Metrikleri
 8. Sonuçlar ve Tartışma
 9. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları
 10. Sonuç ve Gelecekte Yapılabilecek Geliştirmeler
 11. Kaynaklar ve Kullanılan Kütüphaneler
- Ek. Grup İçi Görev Dağılımı

1. Giriş

Bu proje, Bilgisayar Ağları dersinde ele alınan taşıma katmanı kavramlarının —özellikle güvenilir veri aktarımı, hata denetimi ve akış kontrolü— uygulamalı olarak deneyimlenmesi amacıyla geliştirilmiştir. UDP (User Datagram Protocol), bağlantısız ve düşük gecikmeli bir taşıma protokolü olmakla birlikte; teslim garantisi, sıralama veya yinelenme denetimi sağlamaz. Dolayısıyla UDP üzerinden yapılan bir dosya aktarımında paketler kaybolabilir, sırasız gelebilir, tekrarlanabilir veya bozulabilir.

NetProbe, UDP'nin sağlamadığı güvenilirlik garantilerini UYGULAMA KATMANINDA yeniden inşa eden bir istemci–sunucu sistemidir. Sıra numaraları, ACK (onay) mekanizması, timeout temelli yeniden gönderim, yinelenen paket yönetimi ve checksum/hash tabanlı bütünlük denetimi tamamen proje kapsamında tasarlanıp gerçekleştirilmiştir. Sistem ayrıca aktarım sırasındaki ağ olaylarını kaydeder ve throughput, goodput, RTT, kayıp/yeniden gönderim oranı gibi metrikleri üretmek protokol davranışının nicel olarak analiz edilmesini sağlar.

Projenin başlıca öğrenme hedefleri: UDP ile TCP arasındaki temel farkların kavranması; uygulama katmanında güvenilir bir aktarım mekanizmasının tasarlanması; istemci–sunucu mimarisi ve paket temelli bir protokolün gerçekleştirilmesi; ağ performans metriklerinin ölçülmesi ve deneysel sonuçların teknik olarak yorumlanmasıdır. Raporun ilerleyen bölümlerinde sırasıyla problem tanımı, sistem mimarisi, protokol tasarımı, gerçekleştirme detayları, deney ortamı, performans metrikleri ve dört deney senaryosunun sonuçları teknik yorumlarıyla sunulmaktadır.

2. Problem Tanımı

Çözülen temel problem şudur: Bir dosyanın, güvenilirlik garantisi vermeyen UDP üzerinden, paket kaybı/gecikme/bozulma koşulları altında bile istemciden sunucuya EKSİKSİZ ve DOĞRU biçimde aktarılması ve bu aktarım sürecinin ölçülebilir hale getirilmesi. TCP bu güvenilirliği işletim sistemi çekirdeğinde sağlar; bu projede ise benzer güvenceler uygulama katmanında, sıfırdan tasarlanmıştır.

Föyde tanımlanan zorunlu gereksinimler ve bunların sistemde karşılanma biçimi:

Gereksinim	Karşılanma Biçimi
UDP istemci–sunucu	socket (AF_INET, SOCK_DGRAM) ile gerçek UDP soket programlama
Sıra numarası	Her DATA paketinde 4 baytlık seq alanı
ACK mekanizması	Alıcı her geçerli paket için seçici ACK üretir
Timeout kontrolü	Paket başına gönderim zamanı + yapılandırılabilir timeout
Yeniden gönderim	Onaylanmayan paket en fazla 5 kez yeniden gönderilir
Doğru sırada birleştirme	Alıcı seq'e göre tampona yazar, 0..N-1 sırasıyla birleştirir
Duplicate yönetimi	Tekrar gelen paket dosyaya 2. kez yazılmaz; ACK tekrarlanır
Bütünlük denetimi	Paket başına CRC32 + tüm dosya için SHA-256 karşılaştırması
Olay loglama	Gönderim/ACK/timeout/retransmit olayları zaman damgalı CSV
Performans analizi	throughput, goodput, completion time, retransmission rate, RTT

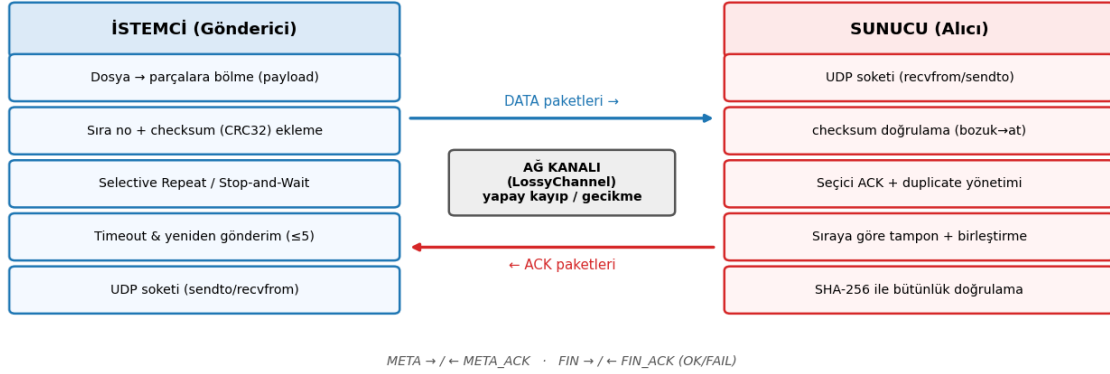
Tablo 1. Zorunlu gereksinimler ve karşılanma biçimleri.

Önemli bir tasarım kısıtı, hazır bir güvenilir aktarım kütüphanesinin kullanılmamasıdır; güvenilirlik mantığının tamamı özgün olarak gerçekleştirilmiştir.

3. Sistem Mimarisi

Sistem iki uç noktadan oluşur: bir dosyayı parçalayıp gönderen İSTEMCİ (gönderici) ve parçaları toplayıp dosyayı yeniden oluşturan SUNUCU (alıcı). İki uç UDP socketleri üzerinden haberleşir. Güvenilirlik tamamen bu iki uçtaki uygulama mantığında sağlanır; aradaki ağ güvenilirmez kabul edilir.

Şekil A. NetProbe sistem mimarisi (istemci-kanal-sunucu)



Şekil A. NetProbe sistem mimarisi: istemci, yapay ağ kanalı ve sunucu.

Yazılım, sorumlulukları net biçimde ayrılmış beş modülden oluşur:

Modül	Sorumluluk
protocol.py	Paket biçimi, serileştirme/çözme, CRC32 checksum, SHA-256 yardımcıları
netstim.py	LossyChannel: yapay paket kaybı/gecikme/bozulma (deney koşulları)
metrics.py	Olay loglama ve performans metriklerinin hesaplanması
server.py	UDP alıcı: ACK üretimi, duplicate yönetimi, birleştirme, bütünlük
client.py	UDP gönderici: pencere yönetimi, timeout, yeniden gönderim

Tablo 2. Modüller ve sorumlulukları.

Veri akışı şu şekildedir: gönderici dosyayı sabit boyutlu payload parçalarına böler, her parçaya sıra numarası ve checksum ekleyerek UDP ile yollar. Alıcı her paketin checksum'unu doğrular, geçerli paketleri sıra numarasına göre bir tampona yerleştirir ve onay (ACK) gönderir. Tüm parçalar alındığında dosya sırayla birleştirilir ve SHA-256 ile bütünlüğü doğrulanır. Deneylerde araya, gönderilen paketlerin bir bölümünü düşüren/geciktiren LossyChannel yerleştirilerek gerçek ağ koşulları taklit edilir.

4. Protokol Tasarımı

Uygulama katmanı protokolü, tüm paketlerde ortak olan 15 baytlık sabit bir başlık kullanır. Başlık ağ bayt sırasında (big-endian) kodlanır:

Alan	Boyut	Açıklama
type	1 bayt	Paket türü (DATA / ACK / META / META_ACK / FIN / FIN_ACK)
seq	4 bayt	Sıra numarası (DATA'da parça no; ACK'te onaylanan no)
total	4 bayt	Toplam parça sayısı
length	2 bayt	payload uzunluğu (bayt)

checksum	4 bayt	type+seq+total+length+payload üzerinden CRC32
payload	değişken	Veri (DATA) veya META için JSON; diğerlerinde boş

Tablo 3. Veri paketi başlık biçimi (toplam başlık 15 bayt).

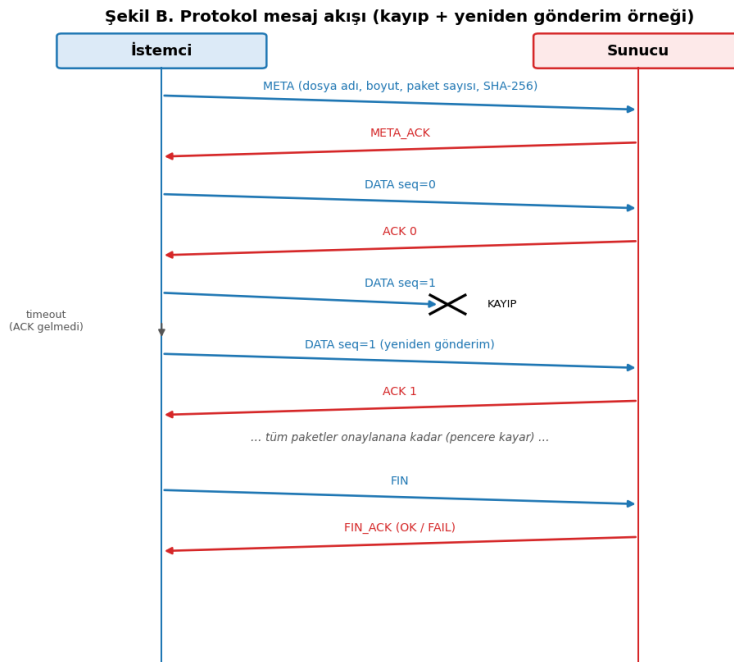
Toplam altı paket türü tanımlanmıştır: META aktarım başında dosya adı, boyut, parça sayısı, payload boyutu, dosyanın SHA-256 özeti ile mod/pencere bilgisini taşır; META_ACK bunun onayıdır. DATA dosya parçalarını, ACK ise alınan parçanın sıra numarasını taşır. FIN aktarımın bittiğini bildirir; FIN_ACK ise sunucunun bütünlük sonucunu (OK/FAIL) geri döndürür.

4.1. Hata Denetimi

Her pakette CRC32 checksum bulunur. Alıcı, paketi aldığı anda checksum'u yeniden hesaplar; tutmuyorsa paket bozuk kabul edilir, YOK SAYILIR ve ACK gönderilmez. Böylece bozuk paket, kaybolmuş paketle aynı şekilde gönderici tarafında timeout'a ve yeniden gönderime yol açar. Dosya geneli bütünlük ise tüm dosyanın SHA-256 özetinin aktarım sonunda karşılaştırılmasıyla garanti altına alınır.

4.2. Güvenilir Aktarım: ARQ Yaklaşımı

Gönderim mantığı tek bir SELECTIVE REPEAT (seçici tekrar) döngüsü olarak tasarlanmıştır ve pencere boyutu ile parametrelendirilir. Pencere = 1 olduğunda davranış STOP-AND-WAIT'e indirgenir (her paket tek tek gönderilip onaylanır); pencere > 1 olduğunda ise SLIDING WINDOW elde edilir (pencere içindeki birden çok paket eşzamanlı uçar, her paketin bağımsız zamanlayıcısı vardır). Bu birleşik tasarım, iki modu aynı kod yolu üzerinden adil biçimde karşılaştırmayı sağlar. Alıcı tarafı moddan bağımsızdır: her geçerli paket için o paketin sıra numarasını taşıyan bir ACK üretir (seçici onay) ve paketleri sıra numarasına göre tamponlar.



Şekil B. Protokol mesaj akışı: META el sıkışması, veri/ACK, kayıp + timeout + yeniden gönderim ve FIN kapanışı.

5. Gerçekleme Detayları

Sistem Python 3 ile, yalnızca standart kütüphane kullanılarak gerçekleştirilmiştir (socket, threading, struct, zlib, hashlib, time, csv, json). Aşağıda kritik mekanizmaların gerçekleme ayrıntıları verilmektedir.

5.1. Gönderici (client.py)

Gönderici, dosyayı payload boyutunda parçalara böler ve her parça için [gönderim_zamanı, deneme_sayısı] bilgisini tutan bir 'inflight' (uçuşta) sözlüğü ile pencereyi yönetir. base değişkeni henüz onaylanmamış en küçük sıra numarasını, next_seq ise gönderilecek bir sonraki paketi gösterir. Her döngü adımında: (a) pencere boşluğu kadar yeni paket gönderilir, (b) en yakın timeout süresine kadar ACK beklenir, (c) süresi dolan paketler yeniden gönderilir. Bir paket için yeniden deneme sayısı 5'i (varsayılan, yapılandırılabilir) aşarsa paket BAŞARISIZ kabul edilir; durum hem kullanıcıya hem log dosyalarına yansıtılır ve aktarım sonlandırılır.

RTT ölçümünde Karn algoritması uygulanmıştır: yalnızca İLK gönderimde onaylanan paketlerden RTT örneği toplanır. Yeniden gönderilmiş bir pakette gelen ACK'in hangi gönderime ait olduğu belirsiz olduğundan, bu örnekler RTT hesabına katılmaz.

5.2. Alıcı (server.py)

Alıcı, gelen geçerli DATA paketlerini sıra numarasına göre tamponlar ve daha önce alınmamışsa kaydeder. Aynı paketin tekrar gelmesi durumunda (duplicate) veri dosyaya İKİNCİ KEZ YAZILMAZ; bunun yerine ilgili ACK tekrar gönderilerek paket yok sayılır. Bu davranış, gönderici tarafında ACK'i kaybolmuş bir paketin yeniden gönderilmesi senaryosunda doğru çalışmayı garanti eder. Tüm parçalar (FIN sonrası) alındığında dosya sırayla birleştirilir, SHA-256 özeti hesaplanıp META'daki değerle karşılaştırılır ve sonuç FIN_ACK ile (OK/FAIL) bildirilir. FIN_ACK kaybına karşı sunucu kısa bir süre daha dinleyerek tekrarlanan FIN'lere yanıt verir.

5.3. Metrikler ve Yapay Kanal

Throughput hat üzerindeki toplam trafiğin (başlıklar ve yeniden gönderimler dahil) süreye oranıdır; **goodput** ise yalnızca yararlı dosya verisinin süreye oranıdır. İkisi arasındaki fark, protokol ek yükü ile yeniden gönderimlerin maliyetini doğrudan görünür kılar. netsim.py içindeki LossyChannel, gönderilen paketlerin bir bölümünü yapılandırılabilir olasılıkla düşürür/geciktirir/bozar ve deneylerde her iki yöne (simetrik) uygulanır.

5.4. Doğruluk Testleri

experiments/test_transfer.py betiği yedi senaryoyu otomatik doğrular ve hepsi başarıyla geçmiştir: (1) Stop-and-Wait kayıpsız, (2) Sliding Window kayıpsız, (3) %10 kayıpta Sliding Window, (4) %20 kayıpta Stop-and-Wait, (5) duplicate yönetimi (alıcının veriyi ikinci kez yazmaması), (6) max_retries aşımında başarısız-paket raporlama ve (7) bozulmuş paketin checksum ile yakalanıp yeniden gönderimle kurtarılması. Tüm başarılı aktarımlarda gönderilen ve alınan dosyanın SHA-256 özetleri birebir aynıdır.

6. Deney Ortamı

Deneyler, istemci ve sunucunun ayrı iş parçacıklarında (thread) GERÇEK UDP soketleri ile 127.0.0.1 (loopback) üzerinden haberleştiği bir koşum düzeneğiyle yürütülmüştür. Loopback, deneylerin tekrar üretilebilir olmasını sağlar; gerçek ağ koşulları ise LossyChannel ile enjekte edilen yapay kayıp ve gecikme (her iki yöne simetrik) üzerinden taklit edilir. Her konfigürasyon 2–3 kez tekrarlanmış, grafiklerde ortalama değerler ve standart sapma (hata çubukları) gösterilmiştir. Koşum ortamı: Linux x86-64, Python 3.10; gönderim ve alım gerçek datagram soketleri üzerinden yapılır.

Ortak (sabit) parametreler ile değişkenler aşağıdaki gibidir:

Senaryo	Değişken parametre	Sabit tutulanlar
S1 — Paket boyutu	payload $\in \{256, 512, 1024, 1400\}$ B	small.bin, kayıp=0.03, timeout=0.08 s

S2 — Timeout	timeout $\in \{5, 15, 40, 100, 300\}$ ms	sliding window, kayıp=0.05, gecikme~3 ms
S3 — Kayıp oranı	kayıp $\in \{0, 0.02, 0.05, 0.1, 0.2, 0.3\}$	small.bin, payload=1024, timeout=0.08 s
S4 — Dosya boyutu	{50 KB, 512 KB, 2 MB}	payload=1024, kayıp=0.03, pencere=32

Tablo 4. Deney senaryoları ve parametreleri. S1, S3, S4'te Stop-and-Wait ile Sliding Window karşılaştırılmıştır.

Test dosyaları üç boyutta hazırlanmıştır: small.bin (50 KB), medium.bin (512 KB) ve large.bin (2 MB). Paket başına en fazla yeniden gönderim sayısı tüm deneylerde 5 olarak alınmıştır (föy gereği varsayılan).

7. Performans Metrikleri

Aktarım sonunda hesaplanan ve raporlanan metrikler şunlardır:

Metrik	Tanım
Tamamlanma süresi	İlk veri gönderiminden FIN_ACK alınana kadar geçen süre (s)
Throughput	Hat üzerindeki toplam trafik (başlık + yeniden gönderim dahil) / süre
Goodput	Yalnızca yararlı dosya verisi (filesize) / süre
Yeniden gönderim oranı	Yeniden gönderim sayısı / toplam DATA gönderimi
Timeout sayısı	Süresi dolduğu için tetiklenen yeniden gönderim olayı sayısı
Ortalama RTT	İlk denemede onaylanan paketlerden ölçülen gidiş-dönüş süresi (Karn)
Duplicate (alıcı)	Alıcıda tekrar gelen, yazılmayan paket sayısı
Başarısız paket	Maksimum denemeye rağmen iletilemeyen paket sayısı

Tablo 5. Performans metriklerinin tanımları.

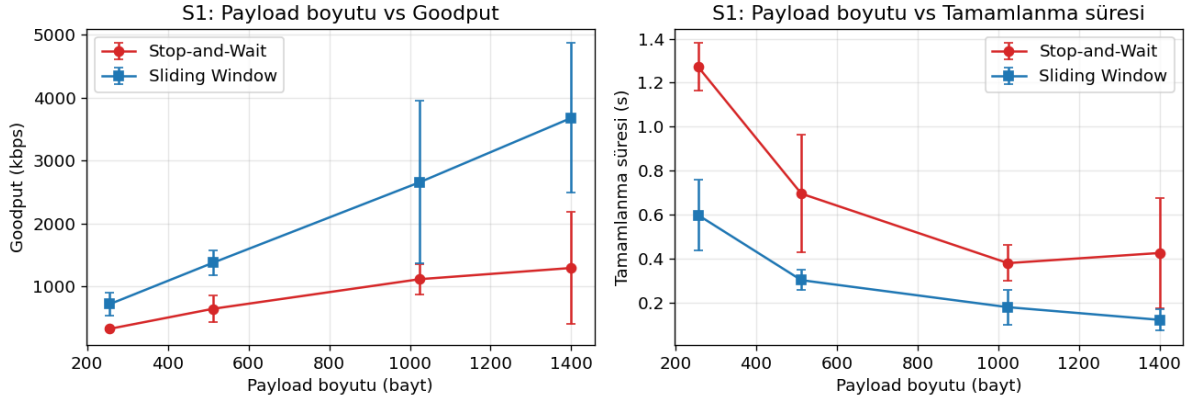
Throughput ile goodput arasındaki fark özellikle önemlidir: kayıp arttıkça yeniden gönderimler nedeniyle hat üzerindeki trafik (throughput) yararlı veriden (goodput) daha hızlı artar; bu fark, güvenilirlik mekanizmasının bant genişliği maliyetini nicel olarak gösterir.

8. Sonuçlar ve Tartışma

Bu bölümde dört deney senaryosunun sonuçları, ölçülen değerlerin protokol davranışıyla ilişkilendirildiği teknik yorumlarla birlikte sunulmaktadır. Tüm değerler tekrarların ortalamasıdır.

8.1. Paket (Payload) Boyutunun Etkisi

Senaryo 1 — Paket (payload) boyutunun etkisi (small.bin, kayıp=0.03)



Şekil 1. Payload boyutuna göre goodput ve tamamlanma süresi.

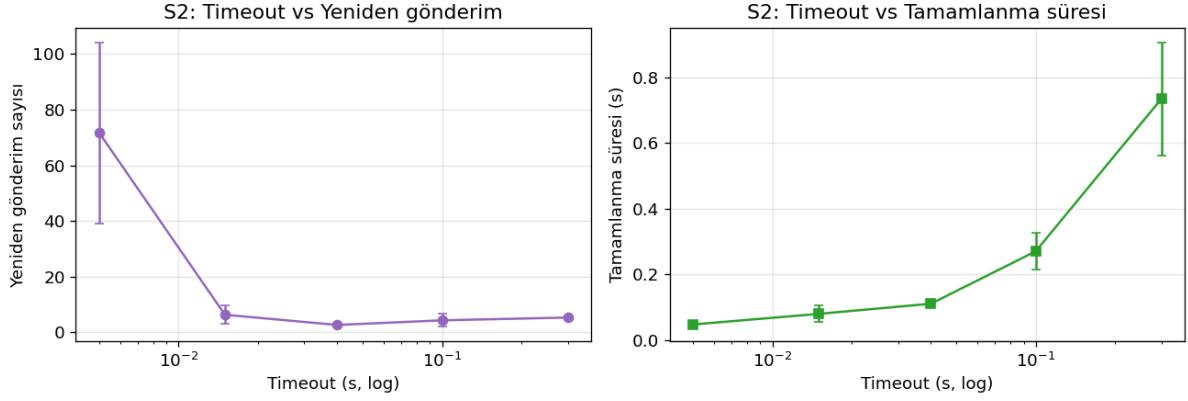
Mod	Payload (B)	Süre (s)	Goodput (kbps)	Yeniden gönderim
Stop-and-Wait	256	1.272	323	9.0
Stop-and-Wait	512	0.695	642	5.3
Stop-and-Wait	1024	0.380	1113	3.0
Stop-and-Wait	1400	0.426	1292	4.0
Sliding Window	256	0.598	716	12.0
Sliding Window	512	0.302	1375	5.7
Sliding Window	1024	0.179	2655	2.3
Sliding Window	1400	0.122	3679	2.3

Tablo 6. Payload boyutu sonuçları (small.bin, kayıp=0.03).

Payload boyutu büyüdükçe dosya daha az sayıda parçaya bölünür; bu da hem paket başına düşen 15 baytlık başlık ek yükünü hem de gereken gidiş-dönüş (ve ACK) sayısını azaltır. Sonuç olarak her iki protokolde de tamamlanma süresi düşer ve goodput artar: Sliding Window'da 256 B'de ~716 kbps olan goodput, 1400 B'de ~3679 kbps'ye (yaklaşık 5 kat) yükselmiştir. Aynı payload'da Sliding Window, Stop-and-Wait'in 2–3 katı goodput sağlar; çünkü pencere mekanizması, bir paketin ACK'ini beklerken diğer paketleri göndererek RTT'yi gizler. Gerçek ağlarda payload'ın MTU'yu (≈ 1500 B) aşması IP parçalanmasına yol açacağından, deneylerde üst sınır 1400 B'de tutulmuştur.

8.2. Timeout Değerinin Etkisi

Senaryo 2 — Timeout değerin etkisi (sliding window, kayıp=0.05, RTT~6-9 ms)



Şekil 2. Timeout değerine göre yeniden gönderim sayısı ve tamamlanma süresi (log eksen).

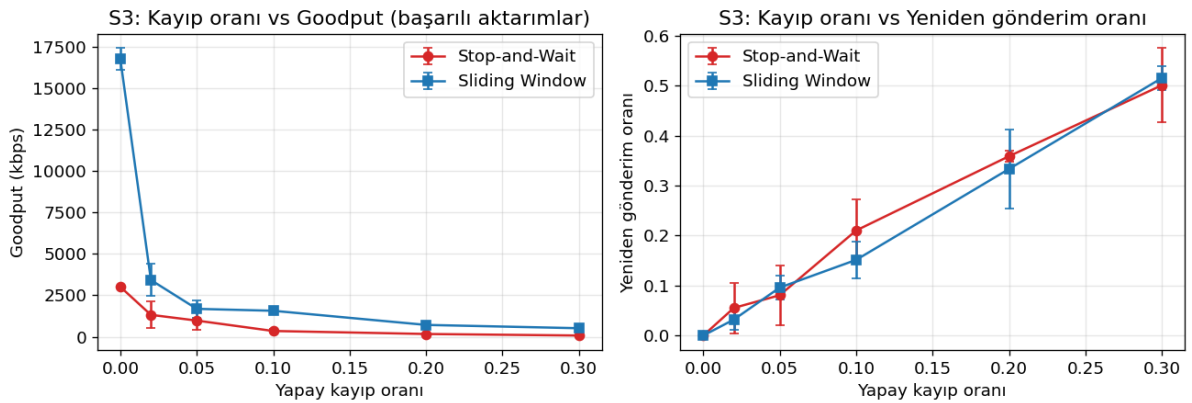
Timeout (ms)	Süre (s)	Goodput (kbps)	Yeniden gönderim	Bütünlük
5	0.057	7518	71.7	%67 (kısmen başarısız)
15	0.081	5520	6.3	%100
40	0.111	3687	2.7	%100
100	0.271	1553	4.3	%100
300	0.734	576	5.3	%100

Tablo 7. Timeout sonuçları (sliding window, kayıp=0.05, RTT~6–9 ms).

Timeout değeri belirgin bir denge (trade-off) ortaya koyar. Timeout RTT'nin altına indiğinde ($5 \text{ ms} < \sim 6\text{--}9 \text{ ms RTT}$), ACK'ler henüz dönmeden paketler 'kayıp' sanılıp gereksiz yere yeniden gönderilir: ortalama 72 yeniden gönderim ve ~ 0.59 yeniden gönderim oranı oluşur. Dahası, bazı paketler 5 denemeyi gecikmiş ACK gelmeden tüketebildiğinden aktarımların üçte biri BAŞARISIZ olmuştur — yani aşırı agresif timeout güvenilirliği tehlikeye atar. Orta değerlerde (15–40 ms, yaklaşık $2\text{--}5 \times \text{RTT}$) yeniden gönderim minimuma (~ 3) iner. Timeout çok büyük seçildiğinde (300 ms) gereksiz yeniden gönderim olmaz; ancak gerçek bir kayıp yaşandığında kurtarma her seferinde tam bir timeout süresi beklediğinden tamamlanma süresi 0.11 s'den 0.73 s'ye fırlar. Sonuç: timeout, RTT'nin küçük bir katı ($\approx 2\text{--}4 \times$) olacak biçimde seçilmelidir.

8.3. Kayıp Oranının Etkisi ve Protokol Karşılaştırması

Senaryo 3 — Kayıp oranının etkisi (small.bin, payload=1024, timeout=0.08)

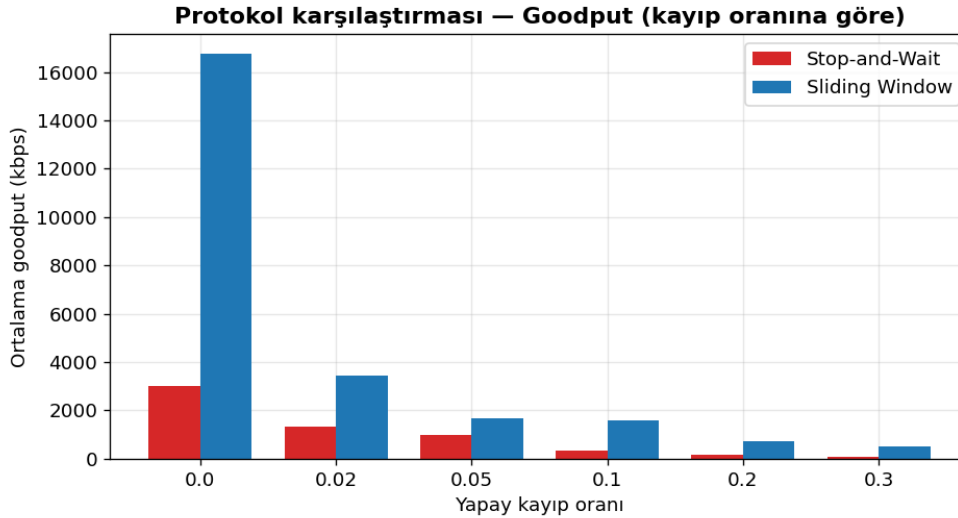


Şekil 3. Kayıp oranına göre goodput ve yeniden gönderim oranı (iki protokol).

Mod	Kayıp	Süre (s)	Goodput (kbps)	Retx oranı	Bütünlük
Stop-and-Wait	0.00	0.136	3010	0.00	%100
Stop-and-Wait	0.05	0.500	979	0.08	%100
Stop-and-Wait	0.10	1.228	352	0.21	%100
Stop-and-Wait	0.20	2.399	171	0.36	%100
Stop-and-Wait	0.30	3.194	214	0.50	%50 (başarısız)
Sliding Window	0.00	0.024	16748	0.00	%100
Sliding Window	0.05	0.259	1686	0.10	%100
Sliding Window	0.10	0.261	1570	0.15	%100
Sliding Window	0.20	0.606	718	0.33	%100
Sliding Window	0.30	0.927	466	0.52	%67 (başarısız)

Tablo 8. Kayıp oranı sonuçları (small.bin, payload=1024, timeout=0.08 s).

Kayıp arttıkça her iki protokolde de goodput düşer ve yeniden gönderim oranı kayıpla yaklaşık orantılı biçimde yükselir (kayıp=0.30'da ~0.50). Ancak iki protokol arasındaki fark çarpıcıdır: Sliding Window her kayıp seviyesinde Stop-and-Wait'in yaklaşık 4–5 katı goodput sağlar (örn. kayıpsızda 16748'e karşı 3010 kbps; kayıp=0.10'da 1570'e karşı 352 kbps). Bunun nedeni, Stop-and-Wait'in HER kayıpta tam bir timeout süresi boyunca durup beklemesi, Sliding Window'un ise kayıp paketi yeniden gönderirken pencere içindeki diğer paketleri göndermeye devam ederek boru hattını dolu tutmasıdır.

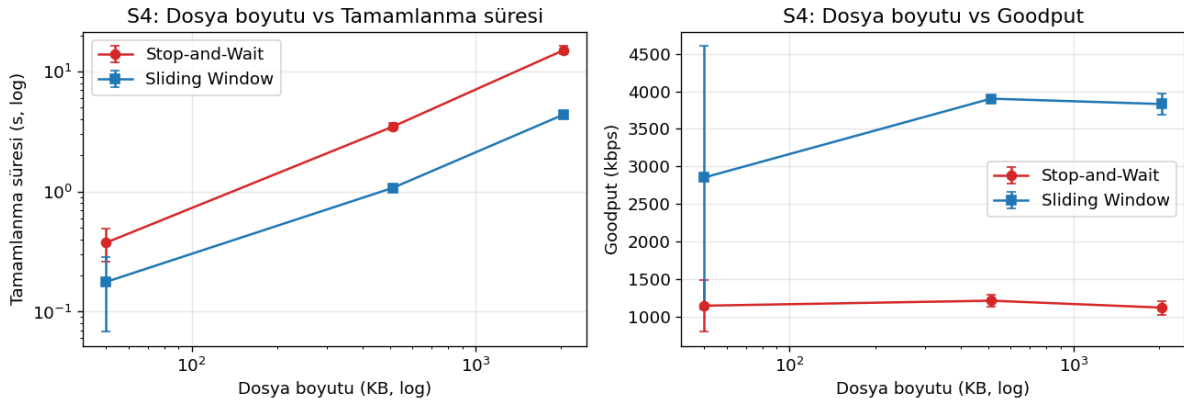


Şekil 4. Goodput açısından protokol karşılaştırması (kayıp oranına göre).

Kayıp=0.30 seviyesinde protokolün pratik sınırına ulaşılır: bazı paketler 5 yeniden denemeye rağmen iletilemediğinden aktarımların bir bölümü başarısız olur (Sliding Window'da koşumların ~%33'ü, Stop-and-Wait'te ~%50'si). Bu, max_retries=5 sınırının çok yüksek kayıpta yetersiz kaldığını gösterir; sınırın artırılması başarı olasılığını yükseltir ancak gecikme pahasına. Bu davranış, güvenilirlik ile gecikme arasındaki temel dengeyi somutlaştırır.

8.4. Dosya Boyutunun Etkisi

Senaryo 4 — Dosya boyutunun etkisi (payload=1024, kayıp=0.03)



Şekil 5. Dosya boyutuna göre tamamlanma süresi (log-log) ve goodput.

Mod	Dosya	Süre (s)	Goodput (kbps)	Yeniden gönderim
Stop-and-Wait	50 KB	0.376	1143	3.0
Stop-and-Wait	512 KB	3.474	1210	26.0
Stop-and-Wait	2 MB	15.067	1117	119.0
Sliding Window	50 KB	0.177	2852	3.0
Sliding Window	512 KB	1.075	3903	29.5
Sliding Window	2 MB	4.382	3831	126.0

Tablo 9. Dosya boyutu sonuçları (payload=1024, kayıp=0.03, pencere=32).

Her iki protokolde de goodput dosya boyutundan büyük ölçüde bağımsız kalır (Sliding Window ~2850–3900 kbps, Stop-and-Wait ~1120–1210 kbps); bu, sistemin büyük dosyalarda verimini koruduğunu gösterir. Ancak Sliding Window'un mutlak zaman avantajı dosya büyüdükçe artar: 2 MB'lık dosyada Stop-and-Wait 15.07 s sürerken Sliding Window yalnızca 4.38 s sürmüştür (≈ 3.4 kat hızlı). Stop-and-Wait her paket için bir RTT beklediğinden paket sayısı ile doğrusal artan bir gecikme biriktirir; Sliding Window ise birden çok paketi eşzamanlı uçurarak bu maliyeti büyük ölçüde ortadan kaldırır. Düşük kayıpta (0.03) tüm aktarımlar bütünlüğü korumuştur.

8.5. Genel Değerlendirme

Deneyler bütününde tutarlı bir tablo ortaya çıkar: Sliding Window, ölçülen tüm koşullarda Stop-and-Wait'e göre belirgin biçimde (tipik olarak 3–5 kat) daha yüksek goodput ve daha düşük tamamlanma süresi sağlar; bu üstünlük RTT ve dosya boyutu büyüdükçe artar. Stop-and-Wait ise daha basit ve anlaşılırdır, fakat her kayıpta durması nedeniyle kayıplı/yüksek gecikmeli ortamlarda hızla verimsizleşir. Performansı belirleyen başlıca ayarlar; payload'ın MTU'ya yakın seçilmesi, timeout'un RTT'nin ~ 2 –4 katı olması ve pencere boyutunun bant genişliği-gecikme çarpımına uygun ayarlanmasıdır.

9. Karşılaşılan Sorunlar ve Çözüm Yaklaşımları

Geliştirme ve deney sürecinde karşılaşılan başlıca teknik sorunlar ve uygulanan çözümler aşağıda özetlenmiştir.

- **Yinelenen (duplicate) paketler:** ACK kaybında veri paketi yeniden gönderilince alıcıya aynı parça ikinci kez ulaşır; verinin yeniden yazılması bozulmaya yol açardı. Çözüm: alıcı, alınan

sıra numaralarını bir küme içinde tutar; tekrar gelen paketi yazmaz, yalnızca ilgili ACK'i tekrar gönderir.

- **Sahte (spurious) yeniden gönderim:** Çok küçük timeout'ta ACK'ler dönmeden paketler kayıp sanılıp gereksiz gönderiliyordu (S2). Çözüm: timeout'un RTT'nin birkaç katı olması gerektiği deneysel olarak gösterildi.
- **RTT belirsizliği:** Yeniden gönderilen pakete gelen ACK'in hangi gönderime ait olduğu belirsizdir. Çözüm: Karn algoritması — RTT yalnızca ilk denemede onaylanan paketlerden ölçülür.
- **FIN_ACK kaybı:** Son onay (FIN_ACK) kaybolursa gönderici FIN'i tekrarlar. Çözüm: sunucu, FIN sonrası kısa süre daha dinleyerek tekrarlanan FIN'lere yanıt verir.
- **Ölçüm doğruluğu (stale dosya):** Deney koşucusunda bütünlük, çıktı dosyası karşılaştırılarak ölçülüyordu; başarısız aktarımda dosya yazılmadığından önceki koşumun dosyası yanlışlıkla 'başarılı' veriyordu. Çözüm: her aktarımdan önce çıktı dosyası silinir ve bütünlük, 'başarısız paket = 0' koşuluyla birlikte değerlendirilir.
- **Aşırı kayıpta başarısızlık:** Çok yüksek kayıpta (≈ 0.30) bazı paketler 5 denemeyi tüketip iletilemedi. Bu bir hata değil, max_retries sınırının bilinçli sonucudur; sistem durumu kullanıcıya ve loglara açıkça yansır.

10. Sonuç ve Gelecekte Yapılabilecek Geliştirmeler

Bu projede, UDP üzerinde uygulama katmanında çalışan, hazır bir kütüphaneye dayanmayan güvenilir bir dosya aktarım sistemi başarıyla tasarlanmış ve gerçekleştirilmiştir. Sistem; sıra numarası, seçici ACK, timeout, en fazla beş yeniden gönderim, yinelenen paket yönetimi ve CRC32/SHA-256 bütünlük denetimi içerir. Hem Stop-and-Wait hem Sliding Window modları gerçekleştirilmiş; dört deney senaryosunda performans ölçülüp karşılaştırılmıştır. Bütün doğruluk testleri geçmiş, başarılı aktarımlarda dosya bütünlüğü SHA-256 ile doğrulanmıştır. Deneyler, Sliding Window'un kayıplı ve gecikmeli ortamlarda Stop-and-Wait'e göre 3–5 kat daha verimli olduğunu nicel olarak ortaya koymuştur.

Gelecekte yapılabilecek geliştirmeler:

- Sabit timeout yerine gözlenen RTT'ye uyarlanan dinamik timeout tahmini (Jacobson/Karels).
- Tıkanıklık denetimi: pencere boyutunu ağ koşullarına uyarlayan mekanizma (yavaş başlangıç vb.).
- Kümülatif ve seçici ACK'i birleştiren (SACK benzeri) daha verimli onay şeması.
- Go-Back-N ile karşılaştırmalı deneyler ve gerçek WAN/laboratuvar testleri.
- Çoklu istemci desteği, isteğe bağlı sıkıştırma veya şifreleme katmanı.

11. Kaynaklar ve Kullanılan Kütüphaneler

- J. F. Kurose, K. W. Ross, Computer Networking: A Top-Down Approach (güvenilir veri aktarımı, Go-Back-N ve Selective Repeat ilkeleri).
- A. S. Tanenbaum, D. J. Wetherall, Computer Networks (taşıma katmanı, ARQ protokolleri).
- RFC 768 — User Datagram Protocol (UDP).
- P. Karn, C. Partridge, "Improving Round-Trip Time Estimates in Reliable Transport Protocols" (RTT belirsizliği için Karn algoritması).
- Python 3 Standart Kütüphane Belgeleri: socket, struct, zlib, hashlib, threading, csv, json.
- Analiz/grafik kütüphaneleri: matplotlib, pandas, numpy (yalnızca sonuç analizi için).

Not: Güvenilir aktarım protokolü ve tüm güvenilirlik mekanizmaları özgün olarak gerçekleştirilmiştir; yukarıdaki kaynaklar kavramsal referans niteliğindedir.

Ek. Grup İçi Görev Dağılımı

Üye	Sorumluluk Alanı
Kerem Özcan	Protokol tasarımı (protocol.py), istemci gönderim mantığı
Hüseyin Acar	Sunucu/alıcı (server.py), duplicate ve bütünlük yönetimi
Kerem Özcan – Hüseyin Acar	Deney koşucusu, analiz/grafikler ve teknik rapor

Tablo 10. Grup içi görev dağılımı